

Report 4 - Turbine Model Family for OpenHPL/OpenHPLjl

Madhusudhan Pandey

19 September 2026

Abstract

This report introduces the turbine model family in OpenHPLjl. The goal is to progress from transparent analytical hydraulic-to-mechanical conversion toward guide-vane, characteristic-map, and turbine-type-specific models while preserving one reusable hydraulic interface. The structure follows the same concise engineering sequence used for the reservoir, rigid-pipe, and surge-tank reports.

1 Updated Context

The turbine is the energy-conversion component between the hydraulic waterway and the rotating shaft:

Reservoir → RigidPipe → SurgeTank → Turbine → Shaft → Generator

.

The waterway determines head and flow. The turbine converts hydraulic power into mechanical power. OpenHPL includes turbine models and supporting turbine documentation, including Francis-turbine formulations and characteristic relationships [1, 2].

2 Generalized Concept Model

The generalized turbine receives hydraulic head and flow and produces mechanical power:

$H, Q, y, \omega \rightarrow \text{Turbine} \rightarrow P_m, T_m$

.

The hydraulic head drop is

$$H = H_{\text{in}} - H_{\text{out}}.$$

The basic hydraulic power is

$$P_h = \rho g Q H.$$

Mechanical power is

$P_m = \eta \rho g Q H$

.

When shaft speed is available,

$$T_m = \frac{P_m}{\omega}.$$

3 Other Models Available in the Literature

A useful hierarchy is

ideal power converter → constant-efficiency turbine → gate-flow model → lookup/Hill-chart model → type-sp

The simplest model computes power from network flow. A gate model adds guide-vane dependence. Higher-fidelity Francis, Pelton, and Kaplan models use turbine-specific nondimensional relationships and characteristic maps. OpenHPL documentation includes turbine characteristic relationships and Francis-turbine modeling resources [1, 2].

4 Mathematics Involved in the OpenHPL Concept/Model

For the first model family,

$$H = H_{\text{in}} - H_{\text{out}}, \quad Q = Q_{\text{in}}, \quad Q_{\text{out}} = -Q.$$

The ideal/constant-efficiency conversion is

$$P_m = \rho g \eta Q H.$$

A first analytical guide-vane approximation is

$$Q = K_q y \sqrt{H},$$

for positive head. Combining the equations gives

$$P_m = \rho g \eta K_q y H^{3/2}.$$

In later models, η and Q become nonlinear functions of guide-vane opening, speed, and head:

$$Q = f_Q(H, \omega, y), \quad \eta = f_\eta(H, \omega, Q, y).$$

5 OpenHPL-Specific Modeling Interpretation

The OpenHPL turbine philosophy is compatible with a progression from simple algebraic conversion toward nonlinear characteristic-map models. The turbine should remain connected hydraulically through the same head/flow ports while the internal constitutive equations become more detailed. This allows a simple turbine to be replaced by a Francis, Pelton, Kaplan, or lookup-table model without redesigning the surrounding waterway.

6 Implementation in Julia ModelingToolkit / SciML

ModelingToolkit supports hierarchical components and acausal connectors. The current package therefore places turbine alternatives under one model-family folder.

```
src/Turbines/  
|-- IdealTurbine.jl  
|-- SimpleGateTurbine.jl  
|-- TurbineLookup.jl # planned  
|-- FrancisTurbine.jl # planned  
|-- PeltonTurbine.jl # planned  
|-- KaplanTurbine.jl # planned
```

The first conversion model is

```
@component function IdealTurbine(  
    name, rho=1000.0, g=9.81, eta=1.0)  
  
    @named inlet = HydraulicPort()  
    @named outlet = HydraulicPort()  
  
    @variables Q(t) H(t) Pm(t)  
  
    eqs = [  
        Q ~ inlet.Q  
        outlet.Q ~ -Q
```

```

    H ~ inlet.H - outlet.H
    Pm ~ rho*g*eta*Q*H
]

System(eqs, t, [Q,H,Pm], [rho,g,eta];
      systems=[inlet,outlet], name=name)
end

```

The first guide-vane model adds

$$Q = K_q y \sqrt{H}.$$

7 Model Assembly and Connections

A turbine can be connected downstream of the hydraulic system:

```

@named tank = SurgeTank()
@named penstock = RigidPipe()
@named turbine = SimpleGateTurbine(y = 0.5)

connections = [
    connect(tank.outlet, penstock.inlet)
    connect(penstock.outlet, turbine.inlet)
]

```

A tail-water boundary is connected to the turbine outlet. A future rotational connector will connect turbine torque to the shaft.

8 Numerical Experiment / Validation

For

$$H = 100 \text{ m}, \quad y = 0.5, \quad K_q = 1,$$

the simple gate law gives

$$Q = 1 \cdot 0.5 \sqrt{100} = 5 \text{ m}^3 \text{ s}^{-1}.$$

With

$$\rho = 1000 \text{ kg m}^{-3}, \quad g = 9.81 \text{ m s}^{-2}, \quad \eta = 0.90,$$

the expected mechanical power is

$$\boxed{P_m = 4.4145 \text{ MW}}.$$

This provides a simple deterministic unit-test target before introducing characteristic maps.

9 Expected Outputs

The minimum turbine outputs are

$$Q(t), \quad H(t), \quad P_m(t),$$

and later

$$\eta(t), \quad T_m(t), \quad \omega(t), \quad y(t).$$

Useful plots include mechanical power versus guide-vane opening, efficiency versus operating point, head-flow response, and turbine characteristic maps.

10 Engineering Interpretation

The turbine family separates model fidelity from system architecture. The surrounding hydropower plant sees the same hydraulic interface while the internal model can evolve from

$$\boxed{\text{ideal} \rightarrow \text{gate-controlled} \rightarrow \text{lookup map} \rightarrow \text{Francis/Pelton/Kaplan}}.$$

This is important for FCR and FREKI studies because model complexity can be increased only where measurements and validation justify it.

Conclusion

The first turbine family establishes the hydraulic-to-mechanical conversion

$$\boxed{P_m = \rho g \eta Q H}$$

and a simple gate-flow relation

$$\boxed{Q = K_q y \sqrt{H}}.$$

The next turbine milestone is a lookup-table/Hill-chart model, followed by a rotational connector and shaft dynamics.

References

- [1] OpenHPL resources, Francis turbine and hydraulic-system formulations. <https://build.openmodelica.org/Documentation/OpenHPL%203.0.1/Resources/Documents/Sharefi2011.pdf>
- [2] OpenHPL turbine-model resource. https://build.openmodelica.org/Documentation/OpenHPL%203.0.1/Resources/Documents/Turbines_model.pdf
- [3] SciML, ModelingToolkit.jl acausal component modeling. https://docs.sciml.ai/ModelingToolkit/stable/tutorials/acausal_components/